

NextGenAI

★ Member-only story

Machine Learning +

Recommendation System +

Tiktok App +

Data Science +

Personalization +

# Content-Based vs Collaborative Filtering: How TikTok and Netflix Hack Your Attention

The Science Behind Addiction in Recommendation Systems



Prem Vishnoi(cloudvala)

Follow

5 min read · Aug 11, 2025

👏 85

💬

↺ 1

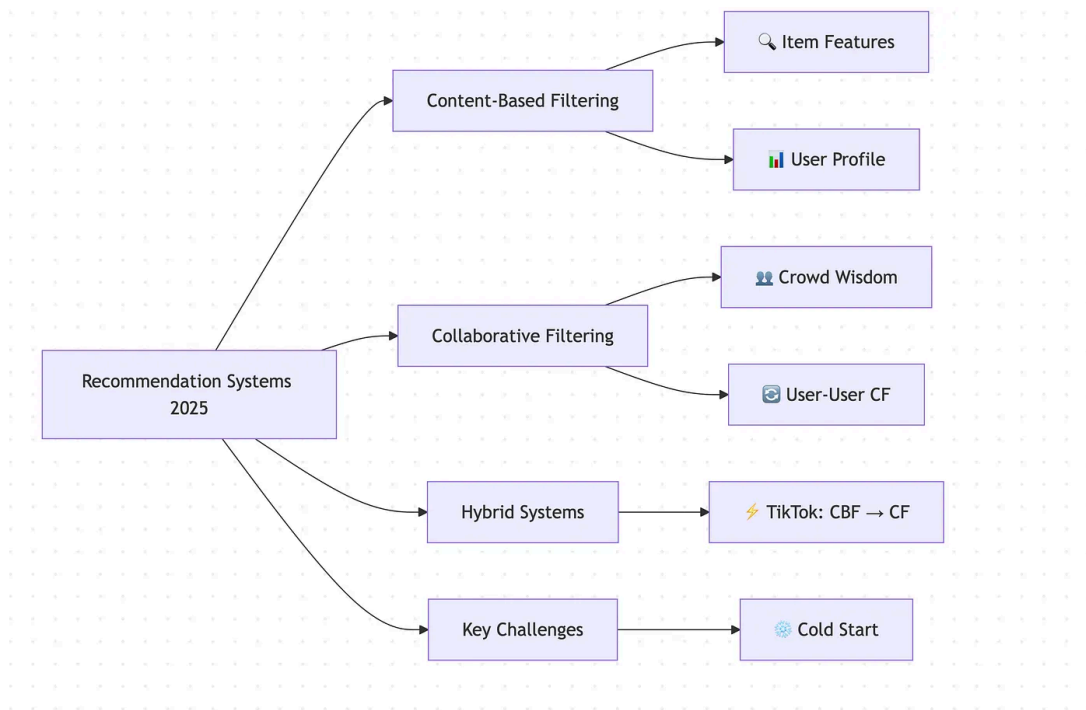
🔖

▶

📄

⋮

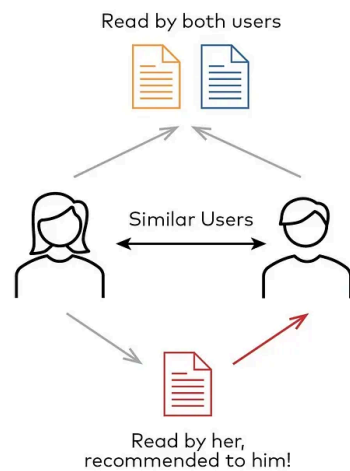




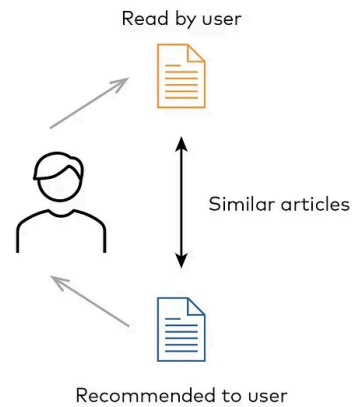
*Illustration: How recommendation systems filter content*

## Introduction:

### COLLABORATIVE FILTERING



### CONTENT-BASED FILTERING



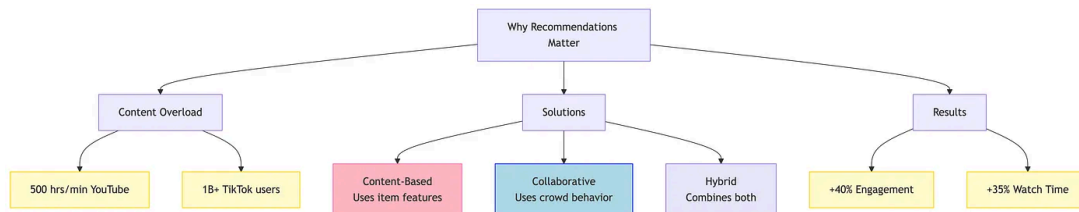
In 2025, recommendation systems power the apps we can't put down  
TikTok's addictive For You page, Netflix's spot on movie picks, Amazon's  
scarily accurate product suggestions.

These systems handle billions of interactions daily, cutting through the noise  
to deliver content you will love. At their core are **Content Based Filtering**

(CBF) and **Collaborative Filtering (CF)**, two approaches that work differently but share one goal: keeping you engaged.

This article breaks down how they work, their pros and cons, and how they solve challenges like the cold start problem.

## 1. Why This Important:



With 500 hours of YouTube content uploaded every minute and TikTok's 1B+ users, finding the right content is a **needle in a haystack problem**.

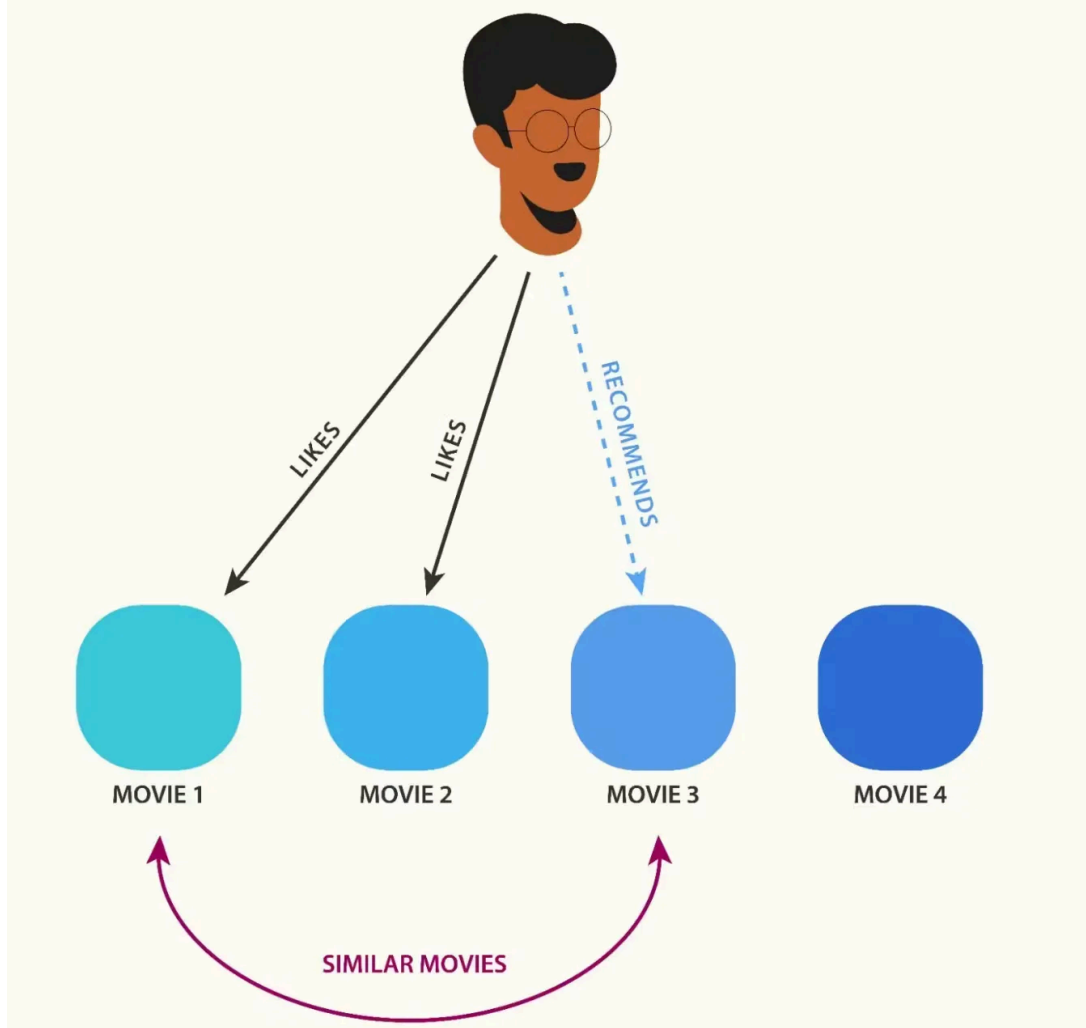
Recommendation systems solve this, boosting engagement by 40% and driving revenue.

CBF uses item features (like video tags), while CF leverages crowd behavior.

Hybrids combine them for maximum impact.

## 2. Content-Based Filtering (CBF):

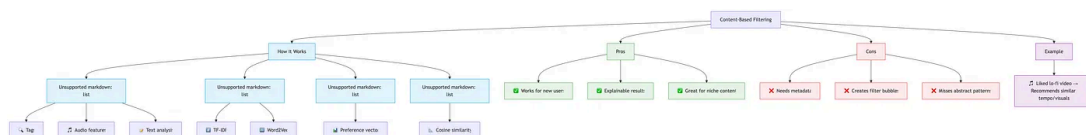
# CONTENT-BASED FILTERING RECOMMENDER SYSTEM



CBF recommends items similar to what you've liked, based on their features like suggesting a lo-fi playlist because you vibed with another.

It recommends based on item features (e.g., genre, keywords) that match your past likes. No crowd needed just you and the item's DNA.

## How It Works CBF



1. **Extract Features:** Pull metadata (eg, TikTok video hashtags, audio tempo).
2. **Build User Profile:** Create a vector of your preferences.
3. **Compute Similarity:** Use cosine similarity to find matches.
4. **Recommend:** Serve the closest items.

[Your Likes] --> [Feature Extraction] --> [Vectorize] --> [Cosine Similarity] -->

### Code Example:

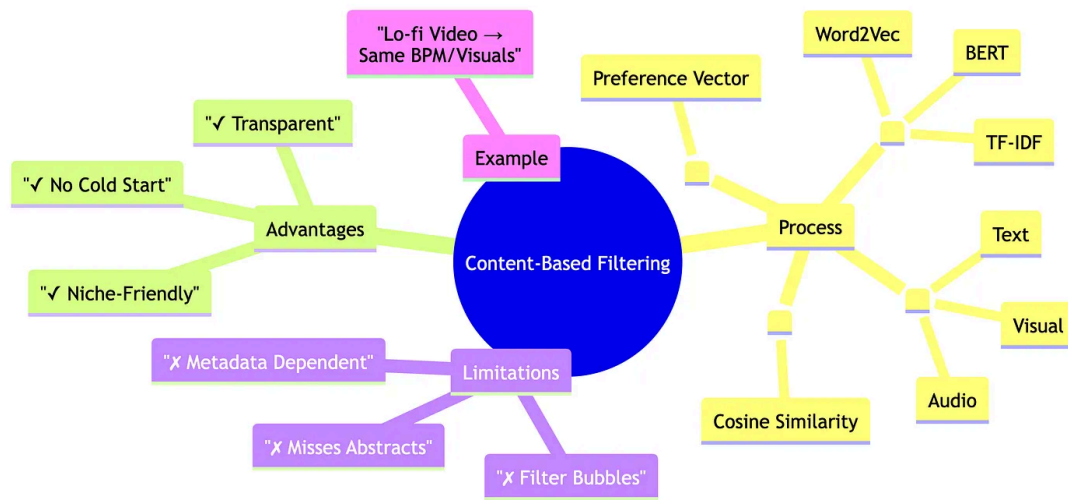
```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Sample movie descriptions
movies = [
    "Sci-fi thriller with mind-bending plot", # Inception
    "Space adventure with time travel",      # Interstellar
    "Romantic comedy in New York"           # When Harry Met Sally
]

# Vectorize
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(movies)

# Similarity between first two movies
sim = cosine_similarity(tfidf_matrix[0:1], tfidf_matrix)
print(f"Similarity between Inception and Interstellar: {sim[0][1]:.2f}") # Outp
```

Similarity between Inception and Interstellar: 0.11



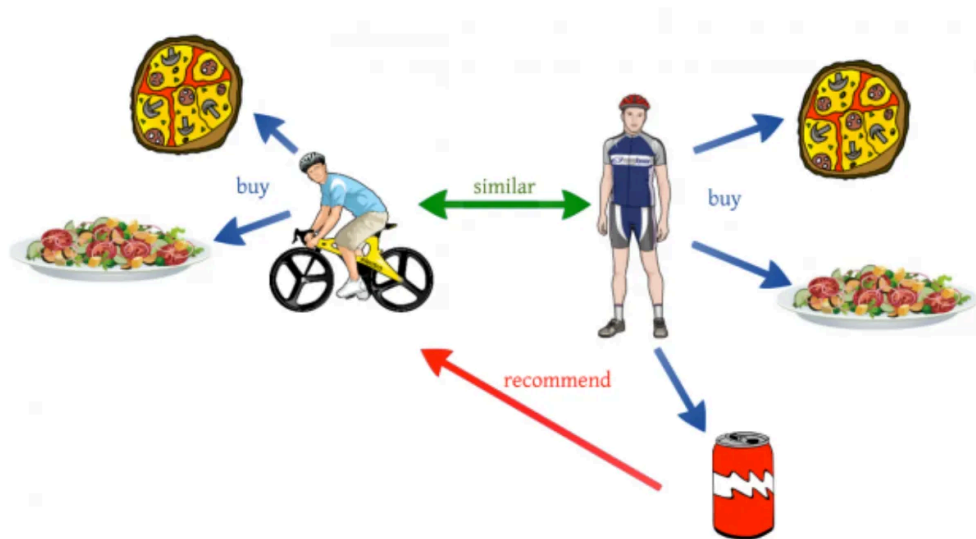
### Pros for CBF:

- Increase engagement and retention
- Works well for **new users** (no need for other users' data)
- Explainable recommendations
- Good for niche content

### Cons for CBF:

- Good for niche content
- Needs **quality metadata**
- Limited novelty — tends to create **filter bubbles**
- Struggles with abstract content (e.g., humor, art)

## 3. Collaborative Filtering (CF)



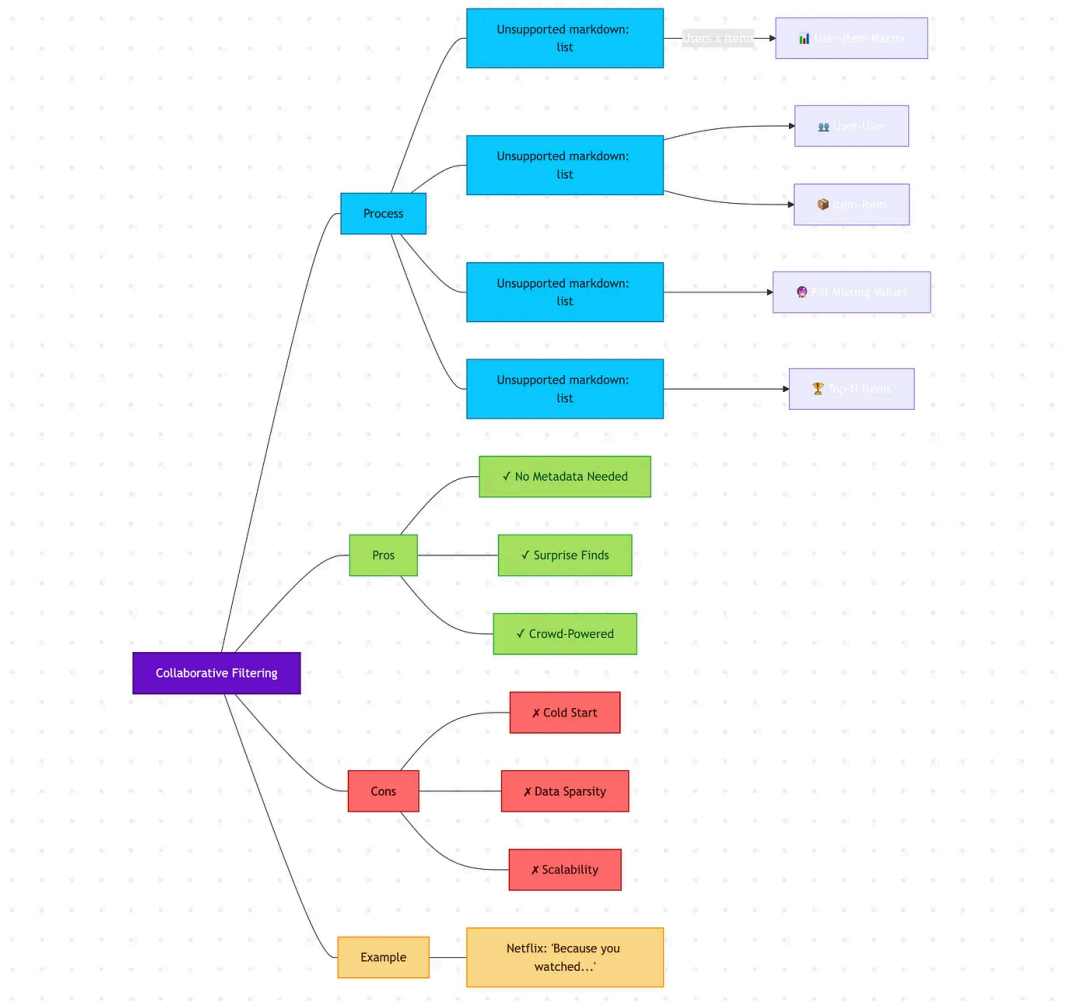
**Core Idea:** Recommend items **based on the behavior of similar users**.

CF suggests items based on what similar users like think Netflix's "Because you watched" feature.

**Two Main Types:**

- **User-Based CF** : Find users like you and recommend what they liked.
- **Item-Based CF**: Find items similar to ones you liked, based on community behavior.

**How It Works CF**



1. **User-Item Matrix:** Track interactions (likes, ratings).
2. **Find Similarities:** Use cosine similarity or Pearson correlation.
3. **Predict:** Fill gaps in the matrix.
4. **Recommend:** Suggest high-scoring items.

### Code Example:

```
import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity

data = pd.DataFrame({
    'Video1': [5, 0, 4],
    'Video2': [4, 3, 0],
    'Video3': [0, 5, 2]
}, index=['UserA', 'UserB', 'UserC'])

similarity = cosine_similarity(data.fillna(0))
sim_df = pd.DataFrame(similarity, index=data.index, columns=data.index)
```

```
print("User-User Similarity Matrix:\n", sim_df.round(2))
# Output: User A and C are similar (0.57)
```



### Pros for CF:

- **User-Based CF** : Find users like you and recommend what they liked.
- No need for item features
- High novelty — can surface unexpected but relevant items
- Works well with unstructured data (music, video)

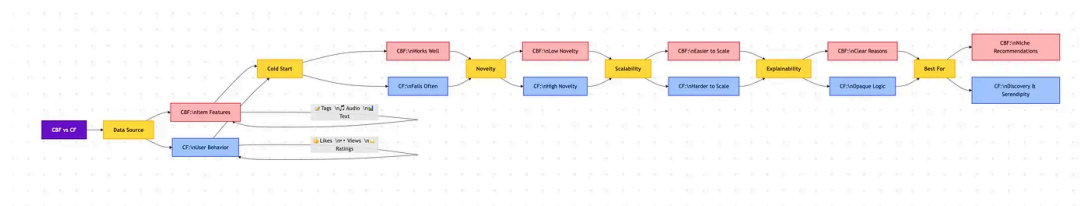
### Cons for CF:

- No need for item features

- Cold start problem for new users/items
- Data sparsity in large systems
- Scalability challenges at **billion-user scale**

| User/Video | Video 1 | Video 2 | Video 3 |
|------------|---------|---------|---------|
| User A     | 5       | 4       | ?       |
| User B     | ?       | 3       | 5       |
| User C     | 4       | ?       | 2       |

#### 4. CBF vs CF at a compare:



#### CBF

- Data Source : Item metadata
- Cold Start : Great for new users/items
- Novelty: Low
- Scalability: Manageable
- Explainability :Clear

#### CF

- Data Source : User interactions
- Cold Start : Struggles with new users/items
- Novelty: High
- Scalability: Challenging

- **Explainability** : Opaque

## 5. The Cold-Start Problem at TikTok Scale:

### Interview Question

*Q How would you handle the cold-start problem for new TikTok users?*

**Answer :**

- **CBF**: Use video features (hashtags, audio embeddings).
- **Popularity Fallback**: Show trending content.
- **CF**: Integrate as interactions grow.
- **Scale**: Use FAISS for fast searches.
- **Diversity**: Balance genres to avoid bias.

New User --> [CBF: Video Tags] --> [Initial Recs] --> [User Interacts] --> [CF +

## 6. Hybrid Systems The Real-World Standard :

Most large platforms combine CBF + CF into **hybrid recommender systems**.

### Hybrid Systems

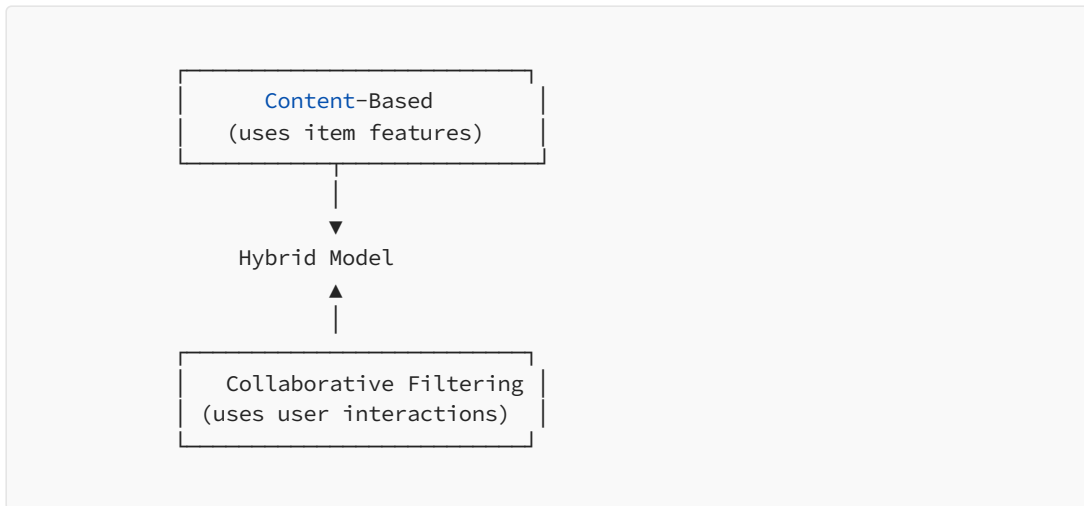
Hybrids combine CBF and CF:

- CBF for new users, CF for veterans.
- Blend scores for accuracy.
- Use neural nets for shared embeddings.

**Example:** TikTok might:

1. Use CBF for new users (video embeddings).

2. Switch to **CF** once enough interactions exist.
3. Blend scores from both models for ranking.



## 7. Scaling to Billions:

- **Matrix Factorization:** Reduce sparsity with SVD.
- **FAISS:** Speed up similarity searches.
- **Reinforcement Learning:** Adapt in real-time.

## 8. The Future: Multi-Modal & RL Systems

Emerging trends in 2025:

- **Reinforcement Learning:** Adapt in real-time.
- **Multi-Modal LLMs:** CLIP-style models analyzing video and text together
- **Reinforcement Learning:** Real-time adaptation

## 8.Final Note:

- Master CBF and CF for interviews.
- Hybrid solves cold-start and boosts engagement.
- Try the code and build your own recommender!
- Next up: “Multi-Modal Recommendation Systems” for TikTok’s secrets.

if you like *Clap, share, and follow me*

Machine Learning +

Recommendation System +

Tiktok App +

Data Science +

Personalization +



## Published in NextGenAI

315 followers · Last published Aug 16, 2026

Follow

Explore the future of AI with tutorials, insights, and projects in Generative AI, Large Language Models, and beyond. From mastering PyTorch and TensorFlow to creating groundbreaking applications, we simplify complex concepts fo



## Written by Prem Vishnoi(cloudvala)

2.3K followers · 104 following

Follow



Head of Data and ML experienced in designing, implementing, and managing large-scale data infrastructure. Skilled in ETL, data modeling, and cloud computing

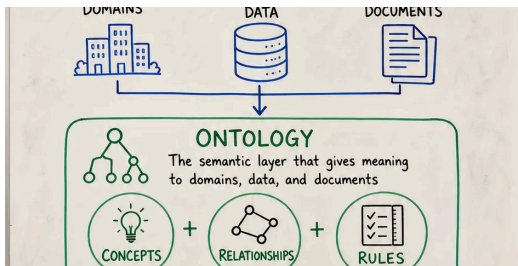
## No responses yet



C J Zorn

What are your thoughts?

## More from Prem Vishnoi(cloudvala) and NextGenAI



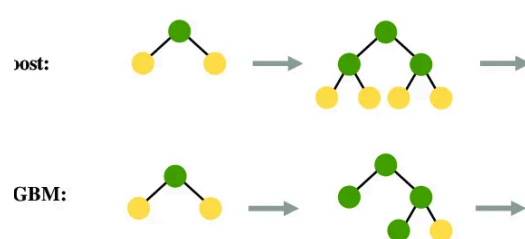
In NextGenAI by Prem Vishnoi(cloudvala) · Aug 7

## Ontology in AI: The Semantic Layer That Connects Domains, Data, an...

AI systems are becoming very good at reading text, answering questions, and...

★ 108 2

💬 ...



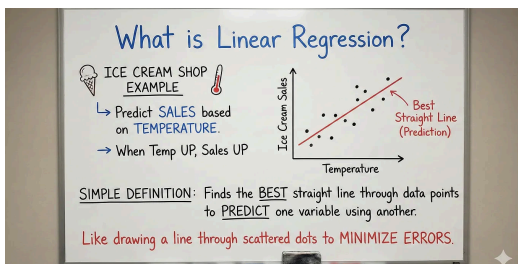
In NextG... by Prem Vishnoi(cloud... · May 23, 2025

## Introduction: XGBoost vs LightGBM — Which One Should Y...

1. Introduction

★ 118

💬 ...



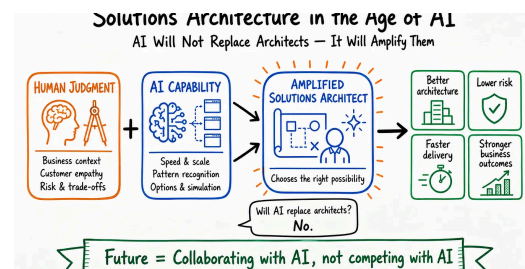
In NextGe... by Prem Vishnoi(cloud... · Dec 4, 2025

## Simple Linear Regression The Complete Guide with Theory,...

From Correlation to Prediction Everything You Need to Know

★ 73 1 2

💬 ...



Prem Vishnoi(cloudvala) · Aug 6

## Solutions Architecture in the Age of AI: Why AI Will Not Replace...

The future belongs to architects who combine human judgment with artificial...

★ 69 1

💬 ...

See all from Prem Vishnoi(cloudvala)

See all from NextGenAI

---

[Redacted]

[Redacted]

[Redacted]

[Redacted]

---

[Redacted]

[Redacted]

[Redacted]

[Redacted]